공격 방어 메커니즘 분석

개념적 배경

왜 공격 방어가 필요한가?

Ethereum 네트워크는 분산 환경에서 운영되므로 다양한 공격 벡터에 노출됩니다. geth는 다음과 같은 공격으로부터 네트워크를 보호해야 합니다:

1. **DoS 공격**: 네트워크 리소스를 고갈시켜 서비스 중단 유발
2. **이클립스 공격**: 노드를 네트워크에서 격리시켜 잘못된 정보 제공
3. **스팸 공격**: 대량의 무효한 트랜잭션으로 네트워크 혼잡 유발
4. **리소스 고갈 공격**: 메모리, CPU, 디스크 등 시스템 리소스 고갈
5. **셀피시 마이닝**: 블록 전파를 지연시켜 불공정한 이익 추구

방어 전략

geth는 다층 방어 전략을 사용합니다:

- **네트워크 레벨**: 연결 제한, 메시지 레이트 리미팅
- **프로토콜 레벨**: 트랜잭션 검증, 가스 제한
- **애플리케이션 레벨**: 리소스 모니터링, 동적 조정

구현 분석

핵심 데이터 구조

1. 연결 스로틀링 (Connection Throttling)

```
const (
// 인바운드 연결 시도를 제한하는 시간
inboundThrottleTime = 30 * time.Second
// 기본 최대 대기 중인 피어 수
defaultMaxPendingPeers = 50
// 다이얼 비율 (아웃바운드 대 인바운드)
defaultDialRatio = 3
// 프레임 읽기 타임아웃
frameReadTimeout = 30 * time.Second
// 프레임 쓰기 타임아웃
frameWriteTimeout = 20 * time.Second
)
```

2. 메시지 레이트 트래커

```
type Tracker struct {
// 특정 데이터 타입에 대한 초당 검색 가능한 아이템 수
capacity  map[uint64]float64
// 피어의 일반적인 응답 지연 시간
roundtrip time.Duration
lock sync.RWMutex
}
```

```
type Trackers struct {
trackers  map[string]*Tracker
// 전체 피어 집합에서 안정적인 RTT 추정값
roundtrip time.Duration
// RTT 추정값의 신뢰도
confidence  float64
// 마지막 튜닝 시간
tuned time.Time
log log.Logger
lock sync.RWMutex
}
```

3. 트랜잭션 검증 구조

```
// 내재적 가스 계산을 위한 매개변수
func  IntrinsicGas(data []byte, accessList types.AccessList, authList []types.Se
isContractCreation, isHomestead, isEIP2028, isEIP3860  bool) (uint64, error)
```

주요 알고리즘

1. 인바운드 연결 스로틀링

```
func (srv *Server) checkInboundConn(remoteIP netip.Addr) error {
if  remoteIP.IsLoopback() {
```

```
return nil
}
// 인터넷 피어의 과도한 연결 시도 거부
now := srv.clock.Now()
srv.inboundHistory.expire(now, nil)
if !netutil.AddrIsLAN(remoteIP) && srv.inboundHistory.contains(remoteIP.String()
return errors.New("too many attempts")
}
// 스로틀링 기록에 추가
srv.inboundHistory.add(remoteIP.String(), now.Add(inboundThrottleTime))
return nil
}
```

2. 메시지 레이트 제한

```
func (t *Tracker) Update(kind uint64, elapsed time.Duration, items int) {
t.lock.Lock()
defer t.lock.Unlock()

// 아무것도 전달되지 않은 경우 (타임아웃/데이터 없음) 처리량을 최소로 감소
if items == 0 {
t.capacity[kind] = 0
return
```

```
}

// 새로운 측정값으로 처리량 업데이트

if  elapsed <= 0 {

elapsed = 1  // 0이 아닌 제수 보장

}

measured := float64(items) / (float64(elapsed) / float64(time.Second))

// 지수 이동 평균으로 용량 업데이트

t.capacity[kind] = (1-measurementImpact)*(t.capacity[kind]) + measurementImpact*

t.roundtrip = time.Duration((1-measurementImpact)*float64(t.roundtrip) + measure

}
```

3. 동적 용량 계산

```
func (t *Tracker) Capacity(kind  uint64, targetRTT time.Duration) int {

t.lock.RLock()

defer  t.lock.RUnlock()


// 실제 측정된 처리량 계산

throughput := t.capacity[kind] * float64(targetRTT) / float64(time.Second)


// 피어가 최소값에 고착되는 것을 방지하기 위해 과대 추정 반환

return  roundCapacity(1 + capacityOverestimation*throughput)

}
```

4. 타임아웃 계산

```
func (t *Trackers) TargetTimeout() time.Duration {
t.tune() // 캐시 업데이트
t.lock.RLock()
defer t.lock.RUnlock()
return t.targetTimeout()
}

func (t *Trackers) targetTimeout() time.Duration {
timeout := time.Duration(ttlScaling * float64(t.roundtrip) / t.confidence)
if timeout > t.OverrideTTLLimit {
timeout = t.OverrideTTLLimit
}
return timeout
}
```

5. 내재적 가스 검증

```
func IntrinsicGas(data []byte, accessList types.AccessList, authList []types.Se
isContractCreation, isHomestead, isEIP2028, isEIP3860 bool) (uint64, error) {
// 기본 트랜잭션 가스 설정
var gas uint64
if isContractCreation && isHomestead {
```

```
gas = params.TxGasContractCreation
} else {
gas = params.TxGas
}
// 데이터 비용 추가
dataLen := uint64(len(data))
if dataLen > 0 {
// 0이 아닌 바이트는 더 비쌈
var nz uint64
for _, byt := range data {
if byt != 0 {
nz++
}
}
// 0이 아닌 바이트와 0 바이트에 대한 가스 계산
if (math.MaxUint64-gas)/params.TxDataNonZeroGas < nz {
return 0, ErrGasUintOverflow
}
gas += nz * params.TxDataNonZeroGas


z := dataLen - nz
if (math.MaxUint64-gas)/params.TxDataZeroGas < z {
return 0, ErrGasUintOverflow
```

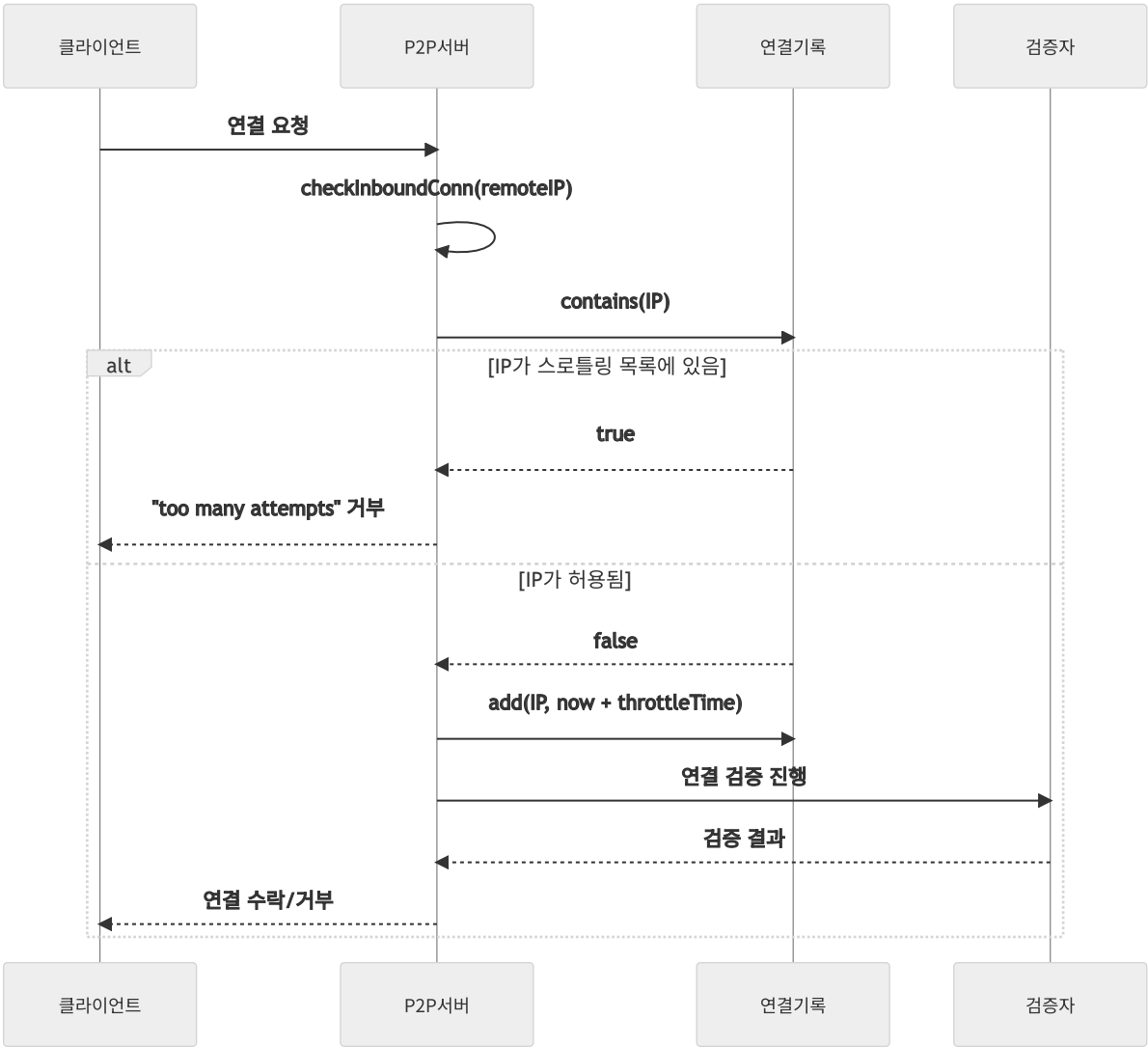
```
}
gas += z * params.TxDataZeroGas
// EIP-3860: 초기 코드 크기 제한
if isContractCreation && isEIP3860 {
lenWords := toWordSize(dataLen)
if (math.MaxUint64-gas)/params.InitCodeWordGas < lenWords {
return 0, ErrGasUintOverflow
}
gas += lenWords * params.InitCodeWordGas
}
}
// 액세스 리스트 비용 추가
if accessList != nil {
gas += uint64(len(accessList)) * params.TxAccessListAddressGas
gas += uint64(accessList.StorageKeys()) * params.TxAccessListStorageKeyGas
}
// 인증 리스트 비용 추가 (EIP-7702)
if authList != nil {
gas += uint64(len(authList)) * params.TxAuthTupleGas
}
return gas, nil
}
```

코드 구조 및 모듈 관계

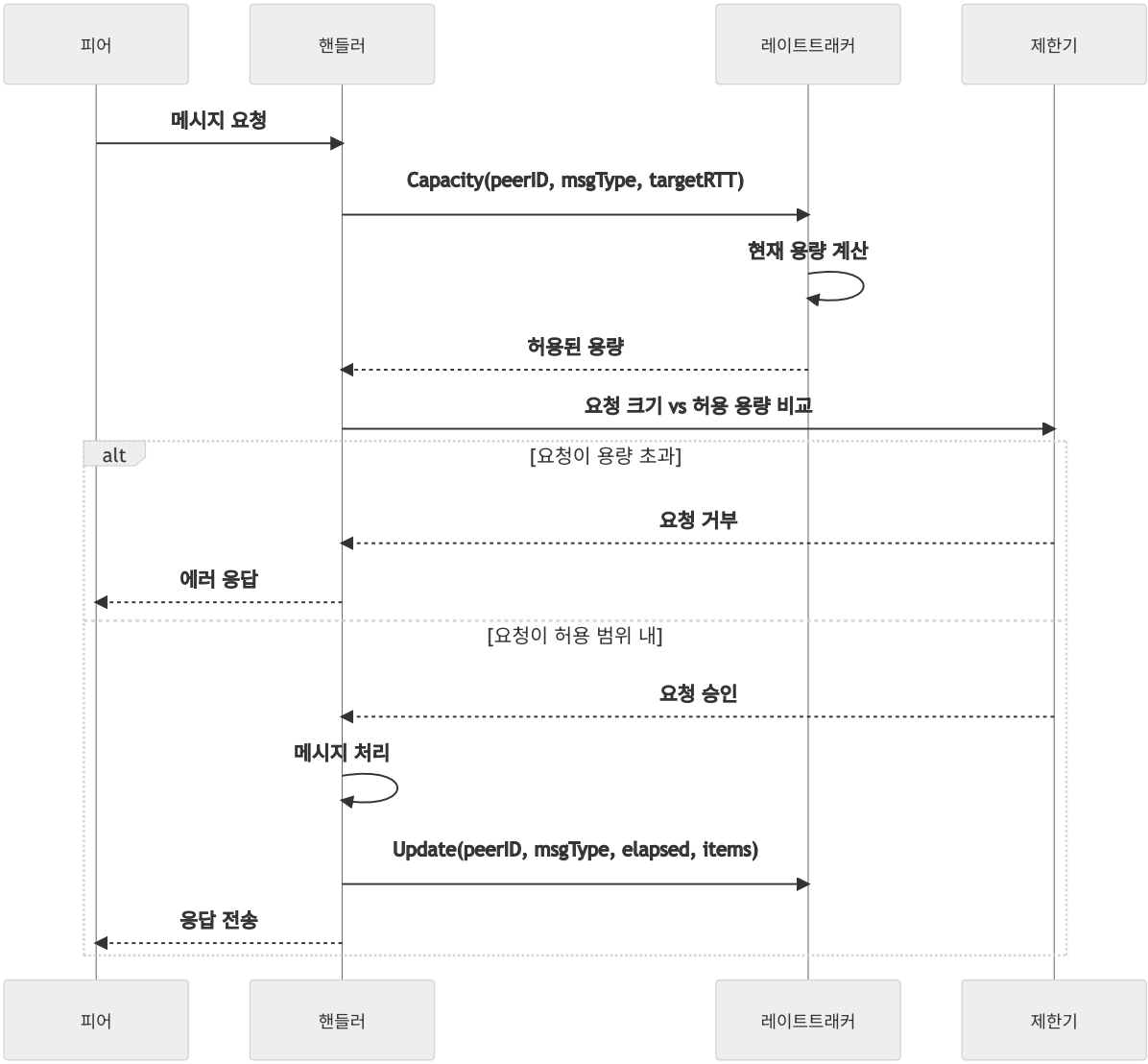
```
보안 방어 시스템/
├── p2p/
| ├── server.go # 연결 관리 및 스로틀링
| ├── msgrate/ # 메시지 레이트 제한
| | └── msgrate.go # 처리량 추적 및 제한
| └── netutil/ # 네트워크 유틸리티
├── core/
| ├── state_transition.go # 트랜잭션 검증
| ├── txpool/ # 트랜잭션 풀 보안
| └── error.go # 보안 관련 에러
├── eth/
| ├── handler.go # 프로토콜 메시지 처리
| └── peerset.go # 피어 관리
└── params/
└── protocol_params.go # 보안 매개변수
```

실행 흐름

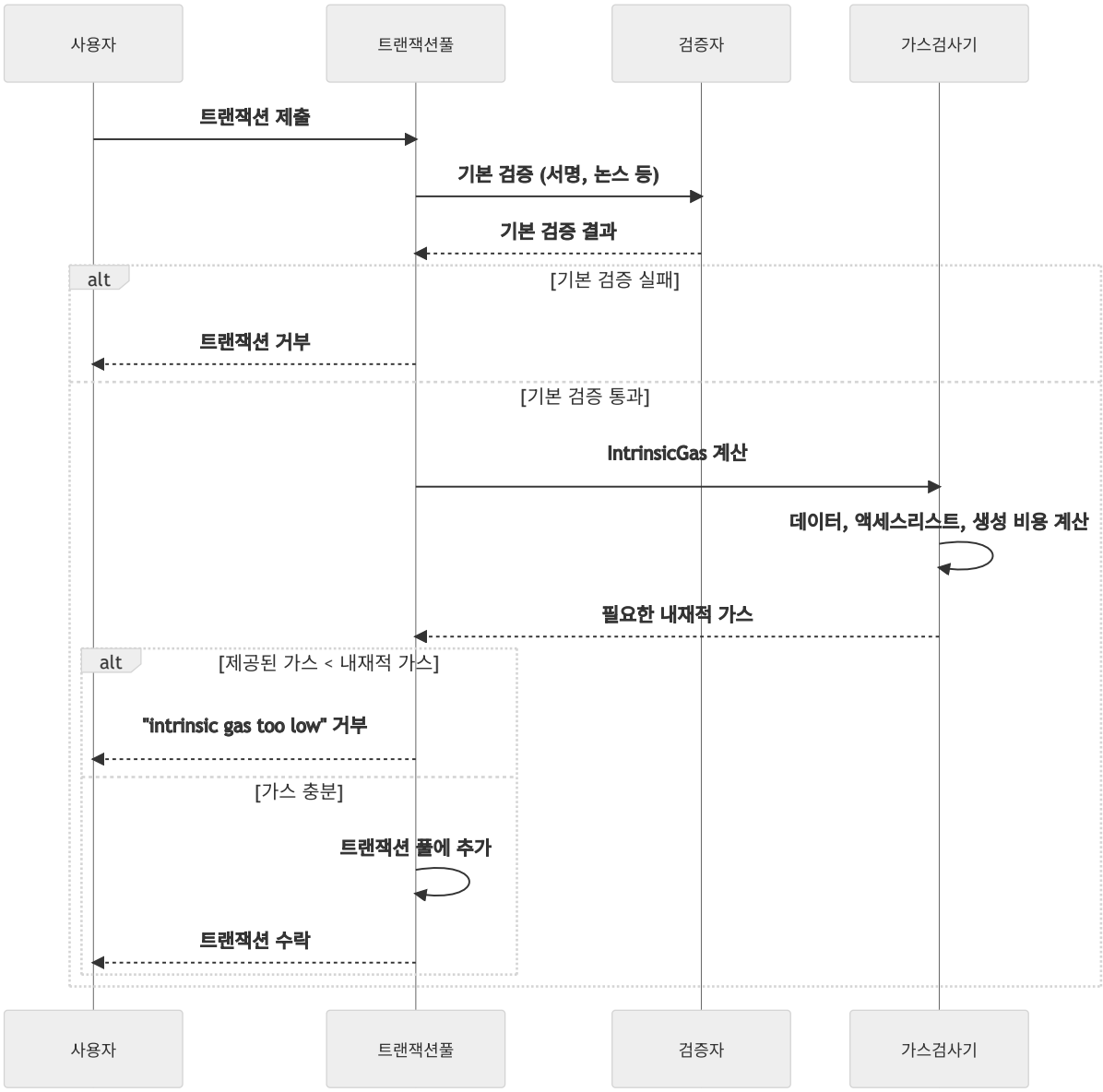
1. 연결 수락 과정



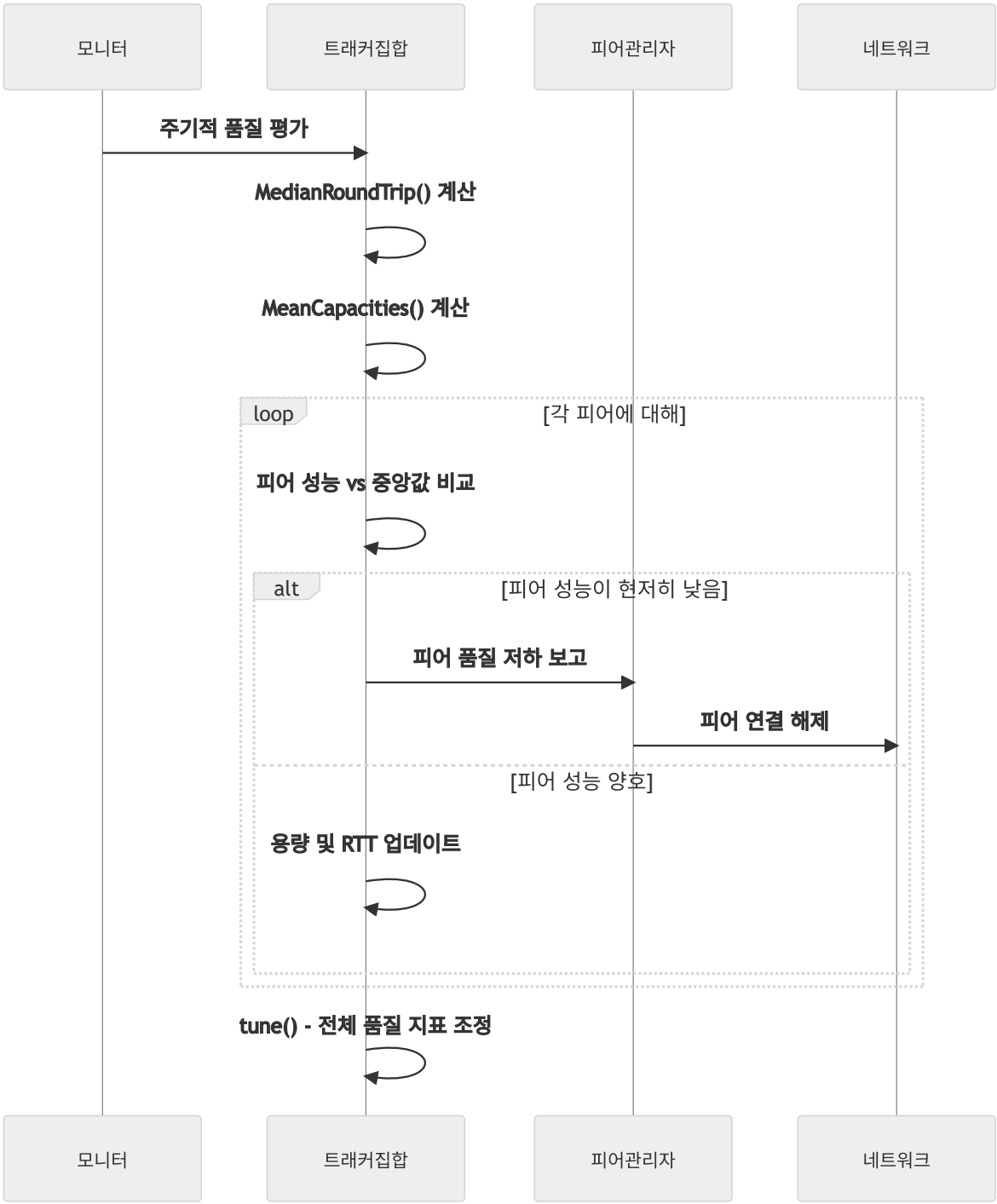
2. 메시지 레이트 제한 과정


피어
핸들러
레이트트래커
제한기
메시지 요청
Capacity(peerID, msgType, targetRTT)
현재 용량 계산
허용된 용량
요청 크기 vs 허용 용량 비교
alt
[요청이 용량 초과]
요청 거부
에러 응답
[요청이 허용 범위 내]
요청 승인
메시지 처리
Update(peerID, msgType, elapsed, items)
응답 전송
피어
핸들러
레이트트래커
제한기

3. 트랜잭션 검증 과정


사용자
트랜잭션풀
검증자
가스검사기
트랜잭션 제출
기본 검증 (서명, 논스 등)
기본 검증 결과
alt
[기본 검증 실패]
트랜잭션 거부
[기본 검증 통과]
IntrinsicGas 계산
데이터, 액세스리스트, 생성 비용 계산
필요한 내재적 가스
alt
[제공된 가스 < 내재적 가스]
"intrinsic gas too low" 거부
[가스 충분]
트랜잭션 풀에 추가
트랜잭션 수락
사용자
트랜잭션풀
검증자
가스검사기

4. 피어 품질 관리 과정


모니터
트래커집합
피어관리자
네트워크
주기적 품질 평가
MedianRoundTrip() 계산
MeanCapacities() 계산
loop
[각 피어에 대해]
피어 성능 vs 중앙값 비교
alt
[피어 성능이 현저히 낮음]
피어 품질 저하 보고
피어 연결 해제
[피어 성능 양호]
용량 및 RTT 업데이트
tune() - 전체 품질 지표 조정
모니터
트래커집합
피어관리자
네트워크

성능 최적화

1. 적응형 레이트 제한

```
const (
// 측정값이 피어의 최종 용량에 미치는 영향
measurementImpact = 0.1
// 용량 과대 추정 비율
capacityOverestimation = 1.01
// RTT 푸시다운 팩터 (더 빠른 요청 유도)
rttPushdownFactor = 0.9
// 최소 RTT 신뢰도
rttMinConfidence = 0.1
)

func (t *Trackers) tune() {
// 동시 업데이트 방지
t.lock.RLock()
dirty := time.Since(t.tuned) > t.roundtrip
t.lock.RUnlock()
if !dirty {
return
}
t.lock.Lock()
defer  t.lock.Unlock()
```

```
if dirty := time.Since(t.tuned) > t.roundtrip; !dirty {
return // 동시 요청이 이미 튜닝 완료
}
// 추정값 업데이트
t.roundtrip = time.Duration((1-tuningImpact)*float64(t.roundtrip) +
tuningImpact*float64(t.medianRoundTrip()))
t.confidence = t.confidence + (1-t.confidence)/2
t.tuned = time.Now()
}
```

2. 메모리 효율적인 히스토리 관리

```
type expHeap []*historyEntry

func (h *expHeap) expire(now mclock.AbsTime, callback func(netip.Addr, time.Tim
for h.Len() > 0 && h.nextExpiry() <= now {
item := heap.Pop(h).(*historyEntry)
if callback != nil {
callback(item.addr, item.time)
}
}
}
```

```
func (h *expHeap) add(addr netip.Addr, exp mclock.AbsTime) {
heap.Push(h, &historyEntry{addr: addr, exp: exp})
// 메모리 사용량 제한
if h.Len() > maxHistoryEntries {
// 가장 오래된 항목 제거
oldest := heap.Pop(h).(*historyEntry)
delete(h.addrMap, oldest.addr)
}
}
```

3. 배치 처리 최적화

```
func (t *Trackers) batchUpdate(updates []measurementUpdate) {
t.lock.Lock()
defer t.lock.Unlock()
// 배치로 업데이트 처리하여 락 경합 감소
for _, update := range updates {
if tracker := t.trackers[update.id]; tracker != nil {
tracker.updateUnsafe(update.kind, update.elapsed, update.items)
}
}
// 배치 완료 후 한 번만 튜닝
if len(updates) > batchTuningThreshold {
```

```
t.tuneUnsafe()

}

}
```

실제 사용 예시

1. 피어 연결 관리

```
func (srv *Server) handleInboundConn(conn net.Conn) {

remoteAddr := conn.RemoteAddr()

remoteIP, err := netip.ParseAddrPort(remoteAddr.String())

if err != nil {

conn.Close()

return

}

// 연결 스로틀링 검사

if err := srv.checkInboundConn(remoteIP.Addr()); err != nil {

srv.log.Debug("Rejected inbound connection", "addr", remoteAddr, "err", err)

conn.Close()

return

}

// 연결 설정 진행

if err := srv.SetupConn(conn, inboundConn, nil); err != nil {
```

```
srv.log.Debug("Failed to setup inbound connection", "addr", remoteAddr, "err", e

}

}
```

2. 메시지 레이트 제한 적용

```
func (h *ethHandler) handleGetBlockHeaders(peer *eth.Peer, msg *eth.GetBlockHead

// 요청 용량 확인

capacity := h.msgrate.Capacity(peer.ID(), eth.BlockHeadersMsg, h.msgrate.TargetR

if  int(msg.Amount) > capacity {

return  fmt.Errorf("requested %d headers, capacity %d", msg.Amount, capacity)

}

start := time.Now()

headers := h.chain.GetBlockHeaders(msg.Origin, msg.Amount, msg.Skip, msg.Reverse

elapsed := time.Since(start)

// 성능 측정 업데이트

h.msgrate.Update(peer.ID(), eth.BlockHeadersMsg, elapsed, len(headers))

return  peer.SendBlockHeaders(headers)

}
```

3. 트랜잭션 검증

```
func  validateTransaction(tx *types.Transaction, chain consensus.ChainHeaderRead
```

```
// 기본 서명 검증
if _, err := types.Sender(signer, tx); err != nil {
return fmt.Errorf("invalid signature: %v", err)
}
// 내재적 가스 검증
intrinsic, err := IntrinsicGas(tx.Data(), tx.AccessList(), tx.SetCodeAuthorizati
tx.To() == nil, true, true, true)
if err != nil {
return err
}
if tx.Gas() < intrinsic {
return fmt.Errorf("intrinsic gas too low: have %d, want %d", tx.Gas(), intrinsi
}
// 가스 가격 검증
if tx.GasPrice().Cmp(chain.Config().MinGasPrice) < 0 {
return fmt.Errorf("gas price too low")
}
return nil
}
```

4. 동적 피어 관리

```
func (pm *PeerManager) managePeerQuality() {
```

```
ticker := time.NewTicker(30 * time.Second)
defer ticker.Stop()
for {
select {
case <-ticker.C:
pm.evaluatePeerPerformance()
case <-pm.quit:
return
}
}
}

func (pm *PeerManager) evaluatePeerPerformance() {
medianRTT := pm.msgrate.MedianRoundTrip()
meanCaps := pm.msgrate.MeanCapacities()
pm.peers.Range(func(id string, peer *Peer) bool {
tracker := pm.msgrate.GetTracker(id)
if tracker == nil {
return true
}
// 성능이 중앙값보다 현저히 낮은 피어 식별
if tracker.roundtrip > medianRTT*2 {
pm.log.Debug("Dropping slow peer", "id", id, "rtt", tracker.roundtrip, "median",
```

```
peer.Disconnect(p2p.DiscUselessPeer)
}
return true
})
}
```

에러 처리 메커니즘

1. 연결 관련 에러

```
var (
errServerStopped = errors.New("server stopped")
errEncHandshakeError = errors.New("rlpx enc error")
errTooManyAttempts = errors.New("too many attempts")
errInboundThrottled = errors.New("inbound connection throttled")
)
```

2. 트랜잭션 검증 에러

```
var (
ErrIntrinsicGas = errors.New("intrinsic gas too low")
ErrGasUintOverflow = errors.New("gas uint64 overflow")
ErrFloorDataGas = errors.New("floor data gas too low")
```

```
ErrInvalidSig = errors.New("invalid transaction v, r, s values")
)
```

3. 메시지 레이트 에러

```
var (
errCapacityExceeded = errors.New("message capacity exceeded")
errTimeoutExceeded = errors.New("request timeout exceeded")
errPeerOverloaded = errors.New("peer overloaded")
)
```

관련 시스템

의존성 관계

1. **p2p**: 네트워크 연결 및 피어 관리
2. **core**: 트랜잭션 검증 및 상태 관리
3. **eth**: 프로토콜 메시지 처리
4. **params**: 보안 매개변수 정의
5. **common**: 공통 유틸리티 및 타입

상호작용 패턴

- **연결 관리**: 새 피어 연결 시 스로틀링 및 검증
- **메시지 처리**: 수신 메시지의 레이트 제한 및 검증

- **트랜잭션 풀**: 새 트랜잭션의 보안 검증
- **피어 평가**: 주기적 피어 성능 평가 및 관리

영향 범위

공격 방어 메커니즘의 변경은 다음 영역에 영향을 미칩니다:

1. **네트워크 성능**: 연결 및 메시지 처리 속도
2. **보안 수준**: 다양한 공격에 대한 저항력
3. **리소스 사용**: CPU, 메모리, 네트워크 대역폭
4. **사용자 경험**: 트랜잭션 처리 시간 및 안정성
5. **네트워크 안정성**: 전체 네트워크의 견고성

이 공격 방어 메커니즘은 Ethereum 네트워크의 보안과 안정성을 보장하는 핵심 구성요소로, 다양한 공격 벡터로부터 네트워크를 보호하고 정상적인 운영을 유지합니다.